

HUBER Q-LEARNING: TD-ERROR-BASED FILTERING FOR MITIGATING OVERESTIMATION BIAS

*Pu Li** *Tao Tan[†]* *Hong Xie[†]* *Xiaoyu Shi** *Mingsheng Shang**

* Chongqing Institute of Green and Intelligent Technology, Chinese Academy of Sciences

[†]School of Computer Science and Technology, University of Science and Technology of China

[‡]CCCC Second Highway Consultants Co., Ltd

ABSTRACT

Value-based reinforcement learning in real-world deployments may encounter intermittently corrupted reward signals, e.g., due to transient sensor failures. We characterize such abnormal rewards using Huber contamination and reveal a previously overlooked failure mode: such contamination can substantially exacerbate the overestimation bias of Q-learning. Abnormally high rewards can induce excessively large positive TD errors, while max-based bootstrapping further propagates the overestimation bias. To address this issue, we propose Huber Q-Learning, a simple yet effective method: clipping the TD error so that it can not exceed a positive threshold. This design offers three benefits: (1) It directly limits the influence of an inflated TD target on the current Q-value; (2) It leaves smaller TD targets unconstrained, allowing them to correct overestimated Q-values.; (3) The threshold can be set conveniently without prior knowledge of the Q-value range. Experiments on both tabular and deep RL benchmarks demonstrate that our method effectively mitigates overestimation bias, yields more accurate value estimates, and achieves higher average scores under Huber contamination than standard baselines.

Index Terms— One, two, three, four, five

1. INTRODUCTION

Value-based reinforcement learning (RL) has become a standard tool for sequential decision-making and now underpins applications ranging from game playing to robotic control and recommender systems[1][2][3]. Among value-based methods, Q-learning[4] is arguably the most fundamental and widely adopted: it is simple, model-free, and converges to the optimal action value under standard assumptions, which has made it the algorithmic core of a large family of deep RL methods. Despite this success, deploying Q-learning in real-world systems remains difficult, and one recurring obstacle lies in the reward signal itself[5].

Real-world deployments rarely provide clean reward signals. Rewards may be collected by physical sensors, annotated by imperfect pipelines, or transmitted over lossy chan-

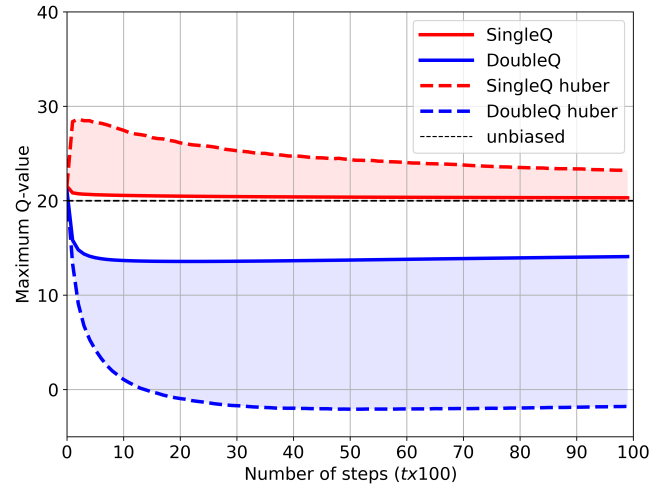


Fig. 1. The maximum Q-value of Q-learning and Double Q-learning under anomalous reward.

nels, all of which can produce intermittent corruption[6]. We model such corruption as Huber contamination[7], in which a small fraction of rewards are drawn from an arbitrary distribution while the remainder follow the intended reward function. Under this model, we find that abnormal reward noise intensifies the classical overestimation bias [8][9] of Q-learning.

Consider a multi-armed bandit setting, which includes one state S with N actions $\{a_1, \dots, a_N\}$. Following previous work [10], each action a_i returns a reward following the correct distribution $\mathcal{N}(1/i, 1^2)$ with probability $1 - \varepsilon$ and the contamination distribution $\mathcal{N}(1/i, 100^2)$ with probability ε . We set the discount factor $\gamma = 0.95$, and the maximum expected Q-value is 20. Our experiments investigate whether the estimation error of Q-learning and Double Q-learning increases in the presence of anomalous reward noise. Fig. 1 shows the maximum Q-value of algorithms after a 10,000-step update. The red solid line represents the estimate of Q-learning when the reward is not affected by anomalous noise ($\varepsilon = 0$), and they are relatively close to the true value of

20. The red dashed line represents the estimate of Q-learning when the reward is affected by anomalous noise ($\varepsilon = 0.01$), and they are relatively far from the true value of 20. The example demonstrates that anomalous reward noise does indeed exacerbate the overestimation bias of Q-learning. The blue line, representing Double Q-learning, exhibits a similar phenomenon of exacerbated bias.

We trace this amplification to an interaction between two mechanisms. Abnormally high rewards directly inflate the temporal-difference (TD) target, and max-based bootstrapping propagates these inflated values while attenuating the low rewards that would correct them. Both effects act through the same max operator, so overestimation accumulates across updates.

To address this, we propose a simple yet effective remedy: clipping the TD error so as not to exceed a positive threshold. The update replaces the standard TD error with its upper-clipped counterpart. This design has two benefits. First, it limits the extent to which an inflated TD target can affect the current estimate, and the threshold is placed on the TD error rather than the Q-value, so it can be set without prior knowledge of the Q-value range. Second, it leaves smaller TD targets unconstrained, so they can still correct overestimated values. The resulting update is a one-line modification to Q-learning and introduces no additional parameters beyond the threshold. Experiments on both tabular and deep RL benchmarks show that our method yields more accurate value estimates under Huber contamination than standard baselines.

Our contributions are as follows:

- We identify a failure mode in which abnormal reward noise exacerbates the overestimation bias of Q-learning, and trace it to the interaction between inflated TD targets and max-based bootstrapping.
- We model abnormal rewards as Huber contamination and propose a simple, one-line modification that clips the TD error from above.
- We show on tabular and deep RL benchmarks that our method yields more accurate value estimates under Huber contamination. The code is at <https://github.com/16061025/HuberQ>.

2. RELATED WORK

Overestimation in Q-learning was first identified and analyzed by Thrun et al. [8], who demonstrated that function approximation errors lead to a systematic positive bias. Existing methods can be broadly categorized into coupling and decoupling paradigms. Coupling methods evaluate the maximum expected Q-value using a single Q-function, always yielding a positive bias. Decoupling methods evaluate the maximum expected Q-value using two independent Q-functions, always yielding a negative bias. **Coupling methods:** Averaged

Q-learning [11] reduces variance by averaging an ensemble of Q-functions, achieving a bias inversely proportional to the ensemble size, yet the bias remains positive with finite ensembles. Mellowmax Q-learning [12] replaces the max operator with a softmax-based operator, enabling continuous bias control via a temperature hyperparameter. Softmax Q-learning [13] and Soft Mellowmax Q-learning [14] further develop softmax-based operators for continuous bias control. Maxmin Q-learning [15] takes the minimum over an ensemble to counteract overestimation, but its bias control is coarse-grained and discrete, limited by the ensemble size. Order Q-learning [16] generalizes ensemble methods via order statistics, though bias control remains discrete.

Decoupling methods: Double Q-learning [9] employs two independent networks, where one selects the greedy action and the other estimates its value, but this often introduces a persistent negative bias that, as noted by [17], can severely slow learning by prematurely discounting high-value states. EBQL [18] extends Double Q-learning to an ensemble of K Q-functions, randomly selecting one for action selection and using the remainder for estimation, yet still suffers from negative bias. AC-CDQ [19] attempts to balance overestimation and underestimation via clipping, but its bias control remains coarse-grained and tied to the number of action candidates. However, current methods focus primarily on the max operation, i.e., the coupling between optimal action selection and value estimation, while neglecting the impact of reward noise on overestimation. The difference between our method and existing methods lies in that we comprehensively consider the combined effect of the max operation and reward noise.

3. HUBER-CONTAMINATED MDP

We consider a reinforcement learning setting formalized as a Markov Decision Process (MDP) [20] with a Huber-contaminated reward signal, which is described by the tuple $(\mathcal{S}, \mathcal{A}, P, \gamma, \mathcal{R}, \mathcal{H}, \varepsilon)$, where $\mathcal{S}$ and $\mathcal{A}$ denote the state and action spaces, $P(s' | s, a)$ gives the transition probability, $\gamma \in (0, 1)$ is the discount factor, and $\{\mathcal{R}, \mathcal{H}, \varepsilon\}$ is the reward model. To model abnormal rewards, we let the observed reward $\tilde{r}_t$ at each time step t come from the true reward distribution $\mathcal{R}$ with probability $1 - \varepsilon$ and from an arbitrary anomalous distribution $\mathcal{H}$ with probability ε :

$$\tilde{r}_t \sim \begin{cases} \mathcal{R}, & \text{with probability } 1 - \varepsilon, \\ \mathcal{H}, & \text{with probability } \varepsilon, \end{cases} \quad (1)$$

where $\varepsilon \in [0, 1)$ controls the contamination rate and $\mathcal{H}$ may deviate substantially from $\mathcal{R}$, capturing intermittent sensor failures and other abnormal reward sources. At each discrete time step t , the agent at state $s_t \in \mathcal{S}$ chooses an action $a_t \sim \pi(\cdot | s_t)$ following a policy π , after which the environment transitions to $s_{t+1} \sim P(\cdot | s_t, a_t)$ and emits a scalar reward $r_t = R(s_t, a_t)$. The objective is to find an optimal

policy π^* that maximizes the expected cumulative discounted return defined over the true reward distribution $\mathcal{D}_r$.

4. METHOD

Noise Effect: In Q-learning, the TD target, $r + \gamma \max_{a'} Q(s', a')$, directly determines the estimated value of $Q(s, a)$. Under abnormal reward interference, (1) the reward r in the TD target may become abnormally large, inflating the TD target and driving $Q(s, a)$ toward an abnormally high value. (2) In contrast, an abnormally low reward is much less effective at correcting such overestimation: once a high reward propagates into the Q-values of other states and actions, $\max_{a'} Q(s', a')$ also becomes abnormally large, so that $r + \max_{a'} Q(s', a')$ remains overestimated.

One Side Constraint: We therefore conclude that in Q-learning, overestimation arises easily but is difficult to correct once it occurs: large TD targets strongly promote overestimation, whereas small TD targets help correct it. Thus, for both the large TD target and the small TD target, we only need to constrain the side with the large target.

TD Error Constraint: Intuitively, a natural remedy is to bound excessively large TD targets. The difficulty, however, is that without prior knowledge of the true Q-value range, it is unclear what bound is appropriate, since the TD target can be either positive or negative. To address this challenge, we shift from bounding the TD target to bounding the TD error, i.e., $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$. The TD error is the step by which the current estimate moves toward the TD target, so imposing a maximum step size δ limits the influence of an excessively large TD target on the estimate. Since decreasing Q-values is more beneficial than increasing them in Q-learning, the step-size threshold is always positive. Its physical meaning is the maximum rate at which Q-values are allowed to increase, which makes δ convenient to choose.

4.1. Tabular Huber Q-learning

We maintain a Q-table Q_t , and construct the behavior policy π_t using the ϵ -greedy policy based on Q_t :

$$a_t = \begin{cases} \text{Sample}(\mathcal{A}) & p = \epsilon, \\ \arg \max_{a \in \mathcal{A}} Q_t(s_t, a) & p = 1 - \epsilon, \end{cases} \quad (2)$$

where $\text{Sample}(\mathcal{A})$ uniformly selects a random action.

At each step t , the agent is at state s_t and select action a_t using π_t . The environment receives the action a_t and then gives the reward r_t and the next state s_{t+1} . This process generates a sampling data (s_t, a_t, r_t, s_{t+1}) . The data will be used to update the Q-functions.

To update the Q estimation $Q_t(s_t, a_t)$, we first calculate the TD error $\Delta(s_t, a_t)$ like Q-learning:

$$\Delta(s_t, a_t) = r_t + \gamma \max_{a'} Q_t(s_{t+1}, a') - Q_t(s_t, a_t). \quad (3)$$

Then We clip $\Delta(s_t, a_t)$ so that it does not exceed the threshold δ , i.e.,

$$\Delta(s_t, a_t) = \min(\delta, \Delta(s_t, a_t)). \quad (4)$$

Finally, we update $Q_t(s_t, a_t)$ as follows:

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t \Delta(s_t, a_t), \quad (5)$$

where α_t is the learning rate satisfying $\sum_t \alpha_t(s, a) = \infty$ and $\sum_t \alpha_t(s, a)^2 < \infty$. Algorithm 1 outlines our Huber Q-learning.

Following the setup and analytical framework of prior work [8], we analyze the overestimation error of Huber Q.

Assumption 1. Let (s, a) precede $(s', \hat{a})$. The independent noise variables $Y_s^{\hat{a}} \sim U(-\omega, \omega)$, with zero mean $E[Y_s^{\hat{a}}] = 0$. The function approximator is modeled as $Q^{\text{approx}}(s, \hat{a}) = Q^{\text{target}}(s, \hat{a}) + Y_s^{\hat{a}}$. The number of actions satisfies $n > 1$. All n actions share the same target Q-value Q^{target} .

Theorem 1 (Existence of optimal δ). Under Assumption 1, consider the clipped target with a constant offset $\delta \geq 0$: $X = \min(\delta + Q^{\text{approx}}(s, a), r_s^a + \gamma \max_{\hat{a}} Q^{\text{approx}}(s', \hat{a}))$. Define the error $Z_s = X - (r_s^a + \gamma \max_{\hat{a}} Q^{\text{target}}(s', \hat{a}))$ and the expected error function $f(\delta) = \mathbb{E}[Z_s]$. Then there exists a $\delta^* \geq 0$ such that $f(\delta^*) = 0$.

To prove Theorem 1, we establish two properties of $f(\delta)$: continuity, i.e., $f(\delta) \in C([0, +\infty))$; opposite signs at end-points, i.e., $f(0) < 0$ and $\lim_{\delta \rightarrow +\infty} f(\delta) > 0$. We then apply the Intermediate Value Theorem to establish existence.

Proof. **Continuity:** Define

$$Z_s(\delta) = X - \left(r_s^a + \gamma \max_{\hat{a}} Q^{\text{target}}(s', \hat{a}) \right) \quad (6)$$

$$= \min(\delta + Q^{\text{approx}}(s, a), B) \quad (7)$$

$$- \left(r_s^a + \gamma \max_{\hat{a}} Q^{\text{target}}(s', \hat{a}) \right), \quad (8)$$

where $B = r_s^a + \gamma \max_{\hat{a}} Q^{\text{approx}}(s', \hat{a})$. For any $\delta_1, \delta_2 \geq 0$, the function $\min(\cdot, \cdot)$ is 1-Lipschitz continuous. Hence

$$|Z_s(\delta_1) - Z_s(\delta_2)| \leq |\delta_1 - \delta_2|. \quad (9)$$

Taking expectations and using the linearity of expectation,

$$|f(\delta_1) - f(\delta_2)| = |E[Z_s(\delta_1)] - E[Z_s(\delta_2)]| \quad (10)$$

$$\leq E[|Z_s(\delta_1) - Z_s(\delta_2)|] \quad (11)$$

$$\leq |\delta_1 - \delta_2|. \quad (12)$$

Hence $f(\delta)$ is 1-Lipschitz continuous on $[0, +\infty)$.

Opposite signs:

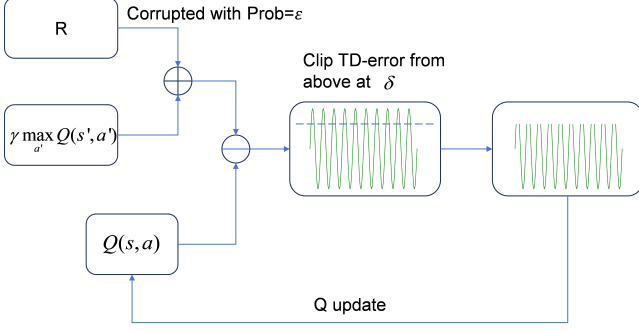


Fig. 2. Huber Q-learning.

case1: $\delta = 0$ Let $T = (r_s^a + \gamma \max_{\hat{a}} Q^{\text{target}}(s', \hat{a}))$

$$f(\delta) = \mathbb{E}[Z_s | \delta = 0] \quad (13)$$

$$= \mathbb{E}[\min(Q^{\text{approx}}(s, a), B) - T] \quad (14)$$

$$\leq \mathbb{E}[Q^{\text{approx}}(s, a) - T] \quad (15)$$

$$= \mathbb{E}[Q^{\text{target}}(s, a) + Y_s^a - T] \quad (16)$$

$$= \mathbb{E}[Q^{\text{target}}(s, a) - (r_s^a + \gamma \max_{\hat{a}} Q^{\text{target}}(s', \hat{a}))] \quad (17)$$

$$+ \mathbb{E}[Y_s^a] \quad (18)$$

$$= 0 \quad (19)$$

case2: $\delta = +\infty$ In this case, Huber Q-learning reduce to Q-learning. Thus,

$$f(\delta) = \mathbb{E}[Z_s | \delta = +\infty] = \gamma \omega \frac{n-1}{n+1} > 0 \quad (20)$$

Since f is continuous on $[0, +\infty)$, $f(0) \leq 0$, and $\lim_{\delta \rightarrow +\infty} f(\delta) > 0$, the Intermediate Value Theorem guarantees the existence of $\delta^* \geq 0$ such that $f(\delta^*) = 0$. $\square$

Algorithm 1 Huber Q-learning

Require: Q-table Q_0 ; TD error upper threshold δ .

- 1: **for** t in $\{0, 1, 2, \dots\}$ **do**
 - 2: Choose action a_t at STATE s_0 by ϵ -greedy policy
 - 3: $r_t, s_{t+1} \leftarrow \text{Env}(s_t, a_t)$
 - 4: $a_{t+1}^* \leftarrow \arg \max_{a' \in \mathcal{A}} Q_t(s_{t+1}, a')$
 - 5: $Y_{t+1} \leftarrow r_t + \gamma Q_t(s_{t+1}, a_{t+1}^*)$
 - 6: $Q_{t+1}(s_t, a_t) \leftarrow Q_t(s_t, a_t) + \alpha \min(\delta, Y_t - Q_t(s_t, a_t))$
 - 7: **end for**
-

4.2. deep extension

We extends Huber Q learning to complex and high-dimensional domains, leads to Huber DQN. We parameterize the Q-function with deep neural networks $Q(s, a; \theta_t)$. And then to

ensure stable learning in deep reinforcement learning setting [21, 22], we use target networks denoted as $Q(s, a; \theta_t^{\text{target}})$, which is delayed copy of the online network. The behavior policy is based on $Q(s, a; \theta_t)$, as in the tabular setting. For any (s_i, a_i, r_i, s_{i+1}) in the sample buffer $\mathcal{B}_{\text{mini}}$, the target Y_i is calculated as follows:

$$Y_i = r_i + \gamma Q(s_{i+1}, a_{i+1}^*; \theta_t^{\text{target}}). \quad (21)$$

We clip Y_i as:

$$Y_i' = \min(Y_i, Q(s_i, a_i; \theta_t) + \delta). \quad (22)$$

The loss function is:

$$J(\theta_t) = \frac{1}{|\mathcal{B}_{\text{mini}}|} \sum_{i \in \mathcal{I}(\mathcal{B}_{\text{mini}})} [Y_i' - Q(s_i, a_i; \theta_t)]^2, \quad (23)$$

where $\mathcal{I}(\mathcal{B}_{\text{mini}})$ is the set of time step indices t for all samples in set $\mathcal{B}_{\text{mini}}$. Algorithm 2 outlines our Huber DQN.

Algorithm 2 Huber DQN

Require: Q Nets Q, Q^{target} with random parameters $\theta_0, \theta_0^{\text{target}}$ for Q learning; upper δ ; replay buffer $\mathcal{B} = \{\}$

Ensure: $Q^*(s, a) \forall s, a$

- 1: **for** t in $\{0, 1, 2, \dots\}$ **do**
 - 2: **if** $t \bmod V == 0$ **then**
 - 3: $\theta_t^{\text{target}} \leftarrow \theta_t$
 - 4: **end if**
 - 5: $a_t \leftarrow \arg \max_a Q(s_t, a; \theta_t)$ {e.g. ϵ -greedy}
 - 6: $r_t, s_{t+1} \leftarrow \text{Env}(s_t, a_t)$
 - 7: $\mathcal{B} \leftarrow \mathcal{B} \cup \{(s_t, a_t, r_t, s_{t+1})\}$ {Store samples in the buffer}
 - 8: $\mathcal{B}_{\text{mini}} \leftarrow$ Sample random minibatch of transitions from $\mathcal{B}$
 - 9: **for** $(s_i, a_i, r_i, s_{i+1}) \in \mathcal{B}_{\text{mini}}$ **do**
 - 10: $a_{i+1}^* \leftarrow \arg \max_{a' \in \mathcal{A}} Q(s_{i+1}, a'; \theta_t)$
 - 11: $Y_i = r_i + \gamma Q(s_{i+1}, a_{i+1}^*; \theta_t^{\text{target}})$
 - 12: $Y_i' = \min(Y_i, Q(s_i, a_i; \theta_t) + \delta)$
 - 13: **end for**
 - 14: minimize $\mathbb{E}_{(s_i, a_i, r_i, s_{i+1}) \in \mathcal{B}_{\text{mini}}} [Y_i' - Q(s_i, a_i; \theta_t)]^2$
 - 15: **end for**
-

5. EXPERIMENTS

5.1. Multi-Armed Bandit

Environment: Following previous work [10], we consider a simple multi-armed bandit setting. It includes one state S with N arms, i.e., $\mathcal{A} = \{a_i | i = 1, 2, \dots, N\}$. For any arm a_i , it yields a reward drawn from a normal distribution $\mathcal{N}(\mu_i, \sigma_R^2)$ with probability $1 - \varepsilon$, and from a contaminated distribution $\mathcal{N}(\mu_i, 100^2)$, where $\mu_i = 1/i$.

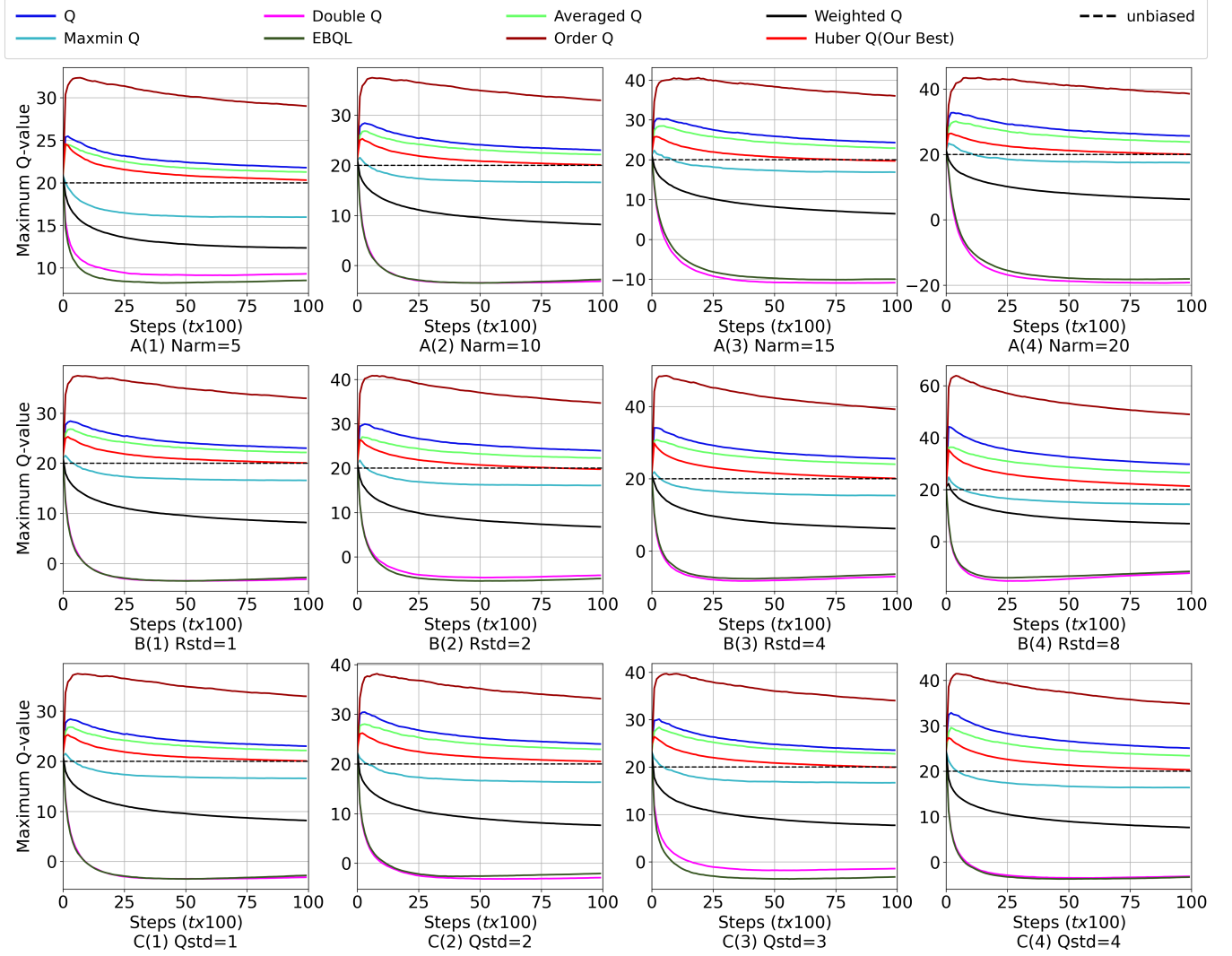


Fig. 3. Maximum Q-value over training steps for algorithms under different settings.

Parameters: Following previous work [10], we set the discount factor $\gamma = 0.95$, the learning rate $\alpha_t(S, a) = 1/[n_t(S, a)^{0.8}]$, the exploration parameter $\epsilon_t(S) = 1/[n_t(S)^{0.5}]$, where $n_t(S)$ and $n_t(S, a)$ are the visited number of state S and state-action pair (S, a) , respectively. Note that under the above setting, the optimal policy always chooses the action a_1 at state S , and the maximum expected Q-value is as $\lim_{T \rightarrow +\infty} \sum_{t=0}^T \gamma^t = 20$. The initial Q-values obey $\mathcal{N}(20, \sigma_Q^2)$, and all results are averaged over 1000 runs.

Baselines: Q-learning [4]; Double Q-learning [9]; Weighted Double Q-learning [10]; Averaged Q-learning [11]; Maxmin Q-learning [15]; Ensemble Bootstrapped Q-learning (EBQL) [18]; Order Q-learning [16].

Performance Comparison: Fig. 3A(1)-(4) shows the Q value estimation performance under different arm number with $N \in \{5, 10, 15, 20\}$. We can observe that a) in all four

cases, our method (red line) yields the estimate closest to the true Q-values; and b) as the number of arms increases, the estimation error of the baseline methods grows larger, while the estimation error of our method remains small. This indicates that our method also achieves effective error suppression as the number of arms varies.

Fig. 3B(1)-(4) shows the Q value estimation performance under different reward variance with $\sigma_R \in \{1, 2, 4, 8\}$. We can observe that a) the estimation error is heavily influenced by the variance of the true rewards; as the reward variance increases, the estimation error of each method increases significantly, whereas the estimation error finally produced by our method remains small. b) In all four cases, our method (red line) consistently yields the estimate closest to the true Q-values. This suggests that our method maintains strong error suppression even under substantial variations in reward

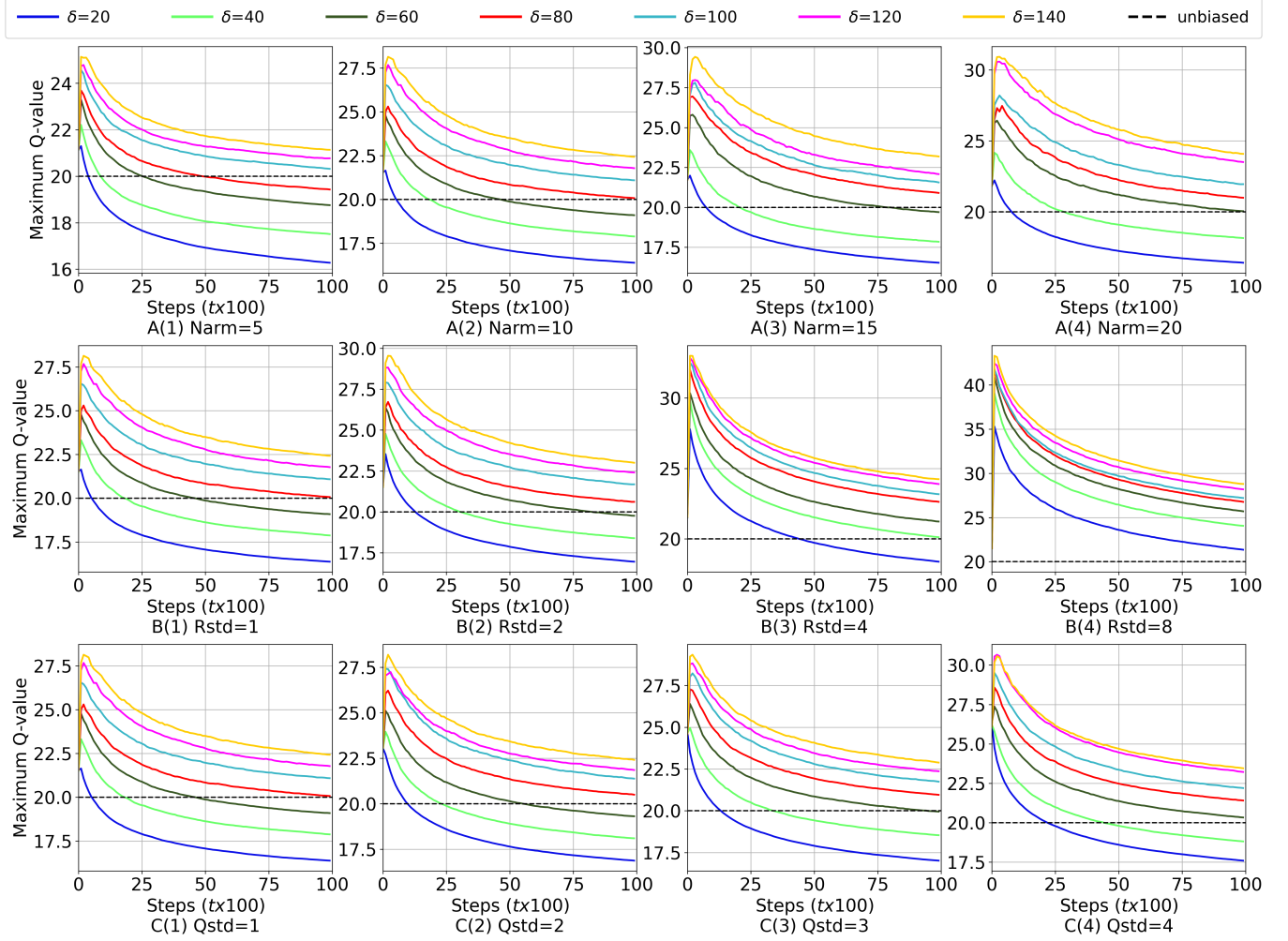


Fig. 4. Maximum Q-value over training steps for Huber Q-learning under different δ .

Table 1. The Maxmin Q-value of different algorithms under different environments. The optimal Q-value in each column are highlighted in bold. The second best result in each column are underlined

Algorithm	N_{arm}				σ_R				σ_Q			
	5	10	15	20	1	2	4	8	1	2	3	4
OrderQ	29.03	32.98	36.04	38.57	32.98	34.68	39.26	49.01	32.98	33.16	34.03	34.85
Q	21.78	23.03	24.30	25.66	23.03	23.94	25.55	29.79	23.03	24.02	23.57	25.09
AveragedQ	21.27	22.17	22.92	23.82	22.17	22.29	24.03	26.57	22.17	22.96	22.80	23.42
MaxminQ	15.97	16.59	16.88	17.50	16.59	16.10	15.36	14.38	16.59	16.34	16.69	16.43
WeightedQ	12.34	8.19	6.45	6.28	8.19	6.78	6.22	6.90	8.19	7.67	7.70	7.65
EBQL	8.52	-2.78	-9.98	-18.09	-2.78	-4.87	-6.41	-11.53	-2.78	-2.05	-3.20	-3.27
DoubleQ	9.29	-3.15	-10.86	-19.20	-3.15	-4.15	-7.08	-12.20	-3.15	-2.88	-1.44	-3.09
Huber Q $\delta = 20$	16.28	16.38	16.53	16.42	16.38	16.96	18.38	21.36	16.38	16.87	17.02	17.59
Huber Q $\delta = 40$	17.51	17.88	17.85	18.16	17.88	18.40	20.12	<u>24.06</u>	17.88	18.10	18.54	<u>18.81</u>
Huber Q $\delta = 60$	18.76	<u>19.09</u>	19.70	20.05	<u>19.09</u>	19.77	<u>21.23</u>	25.70	<u>19.09</u>	<u>19.31</u>	19.95	20.33
Huber Q $\delta = 80$	<u>19.43</u>	20.07	<u>20.90</u>	<u>21.00</u>	20.07	<u>20.62</u>	22.63	26.77	20.07	20.50	<u>20.96</u>	21.40
Huber Q $\delta = 100$	20.32	21.09	21.56	21.96	21.09	21.69	23.17	27.21	21.09	21.38	21.74	22.19
Huber Q $\delta = 120$	20.77	21.78	22.08	23.51	21.78	22.41	23.90	28.19	21.78	21.86	22.37	23.22
Huber Q $\delta = 140$	21.14	22.43	23.19	24.09	22.43	23.00	24.24	28.80	22.43	22.41	22.89	23.45

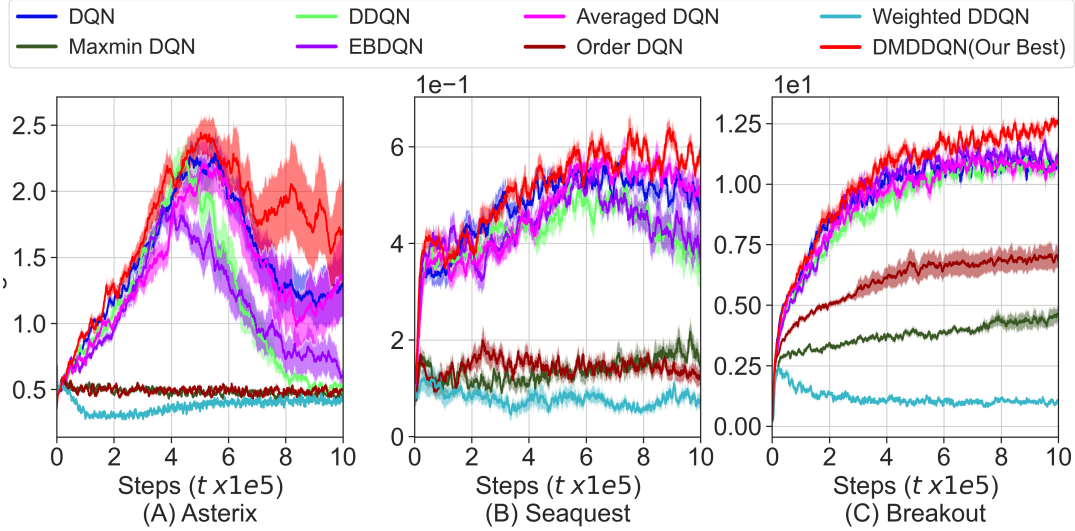


Fig. 5. Average return per episode of baselines under different environments.

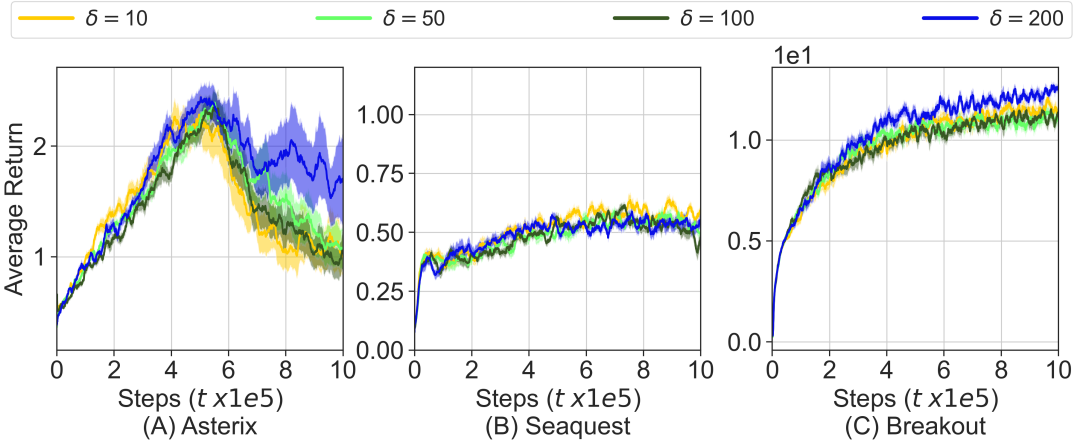


Fig. 6. Average return per episode of Huber DQN under different δ .

variance.

Fig. 3C(1)-(4) shows the Q value estimation performance under different initial Q-values variance with $\sigma_Q \in \{1, 2, 3, 4\}$. We can observe that all algorithms are insensitive to the variance of the initial Q-values, and our algorithm still provides the best Q-value estimates.

Parameter Sensitivity: Under the same settings as the performance comparison experiments, we conducted experiments on the sensitivity of the parameter δ in huberQ, with δ ranging from 20 to 140 in steps of 20. The experimental results are shown in Fig. 4. We can observe that: a) under different conditions, as δ increases, the estimation error of huberQ shifts from underestimation to overestimation; b) for the same δ , as the reward variance increases, the estimation error of huber Q also shifts from underestimation to overestimation; and c) when $\delta = 80$, the estimated values across all

environments are relatively close to the true values, which indicates that finding an appropriate δ is relatively easy.

5.2. Deep Reinforcement Learning

Following previous work [15], this paper selects Asterix, Seaquest, and Breakout environments from MinAtar[23]. The learning rate is $1e-3$, $1e-4$, and $1e-3$ for Asterix, Seaquest, and Breakout, respectively. During training, we add Gaussian noise with a variance of 100^2 to the reward with a probability of 1%. All results are averaged over 20 runs.

Performance Comparison: The result is shown in Fig. 5 and Table 2. In three environments, our method (red) achieves the highest average reward. In the three environments, due to the impact of reward noise, the performance of each algorithm is limited, and some underestimation algorithms such

as MaxMin are greatly affected by noise. This indicates that our algorithm can better resist the impact of abnormal reward noise.

Parameter Sensitivity: To investigate the sensitivity of δ , we evaluate $\delta \in \{10, 50, 100, 200\}$ across three environments. The result is shown in Fig. 6 and Table 2. We observe that: (a) the optimal value of δ varies across environments. Specifically, the optimal value is 200 in Breakout and Asterix, while it is 10 in Seaquest. (b) In Seaquest, the performance varies only marginally as δ increases, whereas in Breakout and Asterix, the performance generally improves as δ increases. This suggests that these two environments are relatively tolerant of overestimation. Future work will investigate how to make δ adaptive across different environments.

Table 2. Average return per episode of baselines under different environments.

Algorithm	Asterix	Seaquest	Breakout
AveragedDQN	1.21	0.52	10.69
WeightedDQN	0.41	0.08	1.00
DDQN	0.52	0.38	10.70
DQN	<u>1.22</u>	0.51	10.82
OrderDQN	0.49	0.13	6.96
EBDQN	0.71	0.41	11.16
MaxminDQN	0.47	0.18	4.42
Huber DQN $\delta = 10$	1.09	0.58	<u>11.50</u>
Huber DQN $\delta = 50$	1.13	<u>0.54</u>	11.05
Huber DQN $\delta = 100$	1.02	0.50	11.09
Huber DQN $\delta = 200$	1.73	0.54	12.34

6. CONCLUSION

We reveal that Huber contamination can substantially exacerbate the overestimation bias of Q-learning, as abnormally high rewards induce excessively large positive TD errors that are further propagated by max-based bootstrapping. We propose Huber Q-Learning, which clips the TD error at a positive threshold, directly limiting inflated TD targets while leaving smaller ones unconstrained to correct overestimated Q-values. Experiments on tabular and deep RL benchmarks show that our method mitigates overestimation bias and achieves higher average scores than standard baselines. Future work includes making the clipping threshold δ adaptive.

7. REFERENCES

- [1] Chen Chu, Yong Li, Jinzhuo Liu, Shuyue Hu, Xuelong Li, and Zhen Wang, “A formal model for multiagent q-learning dynamics on regular graphs,” in *IJCAI*, 2022, pp. 194–200.
- [2] Zhe Zhang, Yukun Zou, Junjie Lai, and Qing Xu, “M2dqn: A robust method for accelerating deep q-learning network,” in *Proceedings of the 2023 15th International Conference on Machine Learning and Computing*, 2023, pp. 116–120.
- [3] Fuqing Zhao, Qiaoyun Wang, and Ling Wang, “An inverse reinforcement learning framework with the q-learning mechanism for the metaheuristic algorithm,” *Knowledge-Based Systems*, vol. 265, pp. 110368, 2023.
- [4] Christopher JCH Watkins and Peter Dayan, “Q-learning,” *Machine learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [5] Yingjie Zhu, Wan Zuha Wan Hasan, Hafiz Rashidi Harun Ramli, Nor Mohd Haziq Norsahperi, Muhamad Saufi Mohd Kassim, and Yiduo Yao, “Deep reinforcement learning of mobile robot navigation in dynamic environment: A review,” *Sensors*, vol. 25, no. 11, pp. 3394, 2025.
- [6] Jingkang Wang, Yang Liu, and Bo Li, “Reinforcement learning with perturbed rewards,” in *Proceedings of the AAAI conference on artificial intelligence*, 2020, vol. 34, pp. 6202–6209.
- [7] Peter J Huber, “Robust estimation of a location parameter,” in *Breakthroughs in statistics: Methodology and distribution*, pp. 492–518. Springer, 1992.
- [8] Sebastian Thrun and Anton Schwartz, “Issues in using function approximation for reinforcement learning,” in *Proceedings of the 1993 connectionist models summer school*. Psychology Press, 2014, pp. 255–263.
- [9] Hado Hasselt, “Double q-learning,” *Advances in neural information processing systems*, vol. 23, 2010.
- [10] Zongzhang Zhang, Zhiyuan Pan, and Mykel J Kochenderfer, “Weighted double q-learning,” in *IJCAI*, 2017, pp. 3455–3461.
- [11] Oron Anschel, Nir Baram, and Nahum Shimkin, “Averaged-dqn: Variance reduction and stabilization for deep reinforcement learning,” in *International conference on machine learning*. PMLR, 2017, pp. 176–185.
- [12] Kavosh Asadi and Michael L Littman, “An alternative softmax operator for reinforcement learning,” in *International Conference on Machine Learning*. PMLR, 2017, pp. 243–252.
- [13] Zhao Song, Ron Parr, and Lawrence Carin, “Revisiting the softmax bellman operator: New benefits and new perspective,” in *International conference on machine learning*. PMLR, 2019, pp. 5916–5925.
- [14] Yaozhong Gan, Zhe Zhang, and Xiaoyang Tan, “Stabilizing q learning via soft mellowmax operator,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, vol. 35, pp. 7501–7509.

- [15] Qingfeng Lan, Yangchen Pan, Alona Fyshe, and Martha White, “Maxmin q-learning: Controlling the estimation bias of q-learning,” *arXiv preprint arXiv:2002.06487*, 2020.
- [16] Tao Tan, Hong Xie, and Defu Lian, “Adaptive order q-learning,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 4946–4954.
- [17] Patigül Abliz and Shi Ying, “Underestimation estimators to q-learning,” *Information Sciences*, vol. 607, pp. 173–185, 2022.
- [18] Oren Peer, Chen Tessler, Nadav Merlis, and Ron Meir, “Ensemble bootstrapping for q-learning,” in *International conference on machine learning*. PMLR, 2021, pp. 8454–8463.
- [19] Haobo Jiang, Jin Xie, and Jian Yang, “Action candidate based clipped double q-learning for discrete and continuous action tasks,” in *Proceedings of the AAAI conference on artificial intelligence*, 2021, vol. 35, pp. 7979–7986.
- [20] Xinyue Chen, Che Wang, Zijian Zhou, and Keith Ross, “Randomized ensembled double q-learning: Learning fast without a model,” *arXiv preprint arXiv:2101.05982*, 2021.
- [21] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [22] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al., “Human-level control through deep reinforcement learning,” *nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [23] Qingfeng Lan, “Gym compatible games for reinforcement learning,” <https://github.com/qlan3/gym-games>, 2019.